



std::simd Without Compromise:

SIMD in C++26 and Beyond, as Fast as Silicon Allows

DANIEL TOWNER &
RUSLAN ARUTYUNYAN



20
26

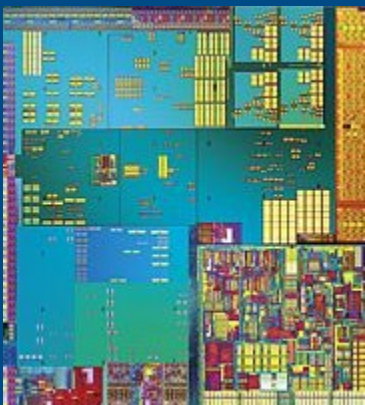


Motivation for `std::simd`

- For 3 decades or more, processors have delivered ever-increasing parallel capability
 - - not through threading, but through data-parallelism
- Data-parallelism now surrounds us, from phones to cloud compute.
- A myriad of C++ libraries support data parallelism, but C++ has lacked a standard, portable vocabulary for expressing it...
-until now



Agenda



What is hardware SIMD?

```
auto f(vec<float> x) {  
    ... return reduce(x + 1.0f);  
}
```

What is `std::simd`?



What is xvec?



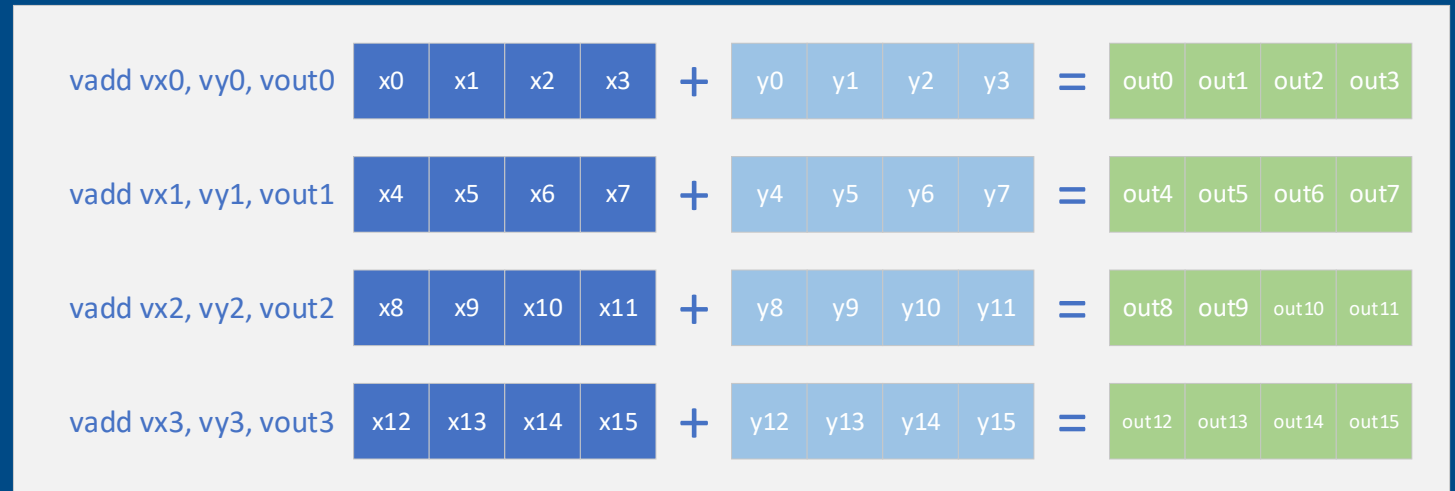
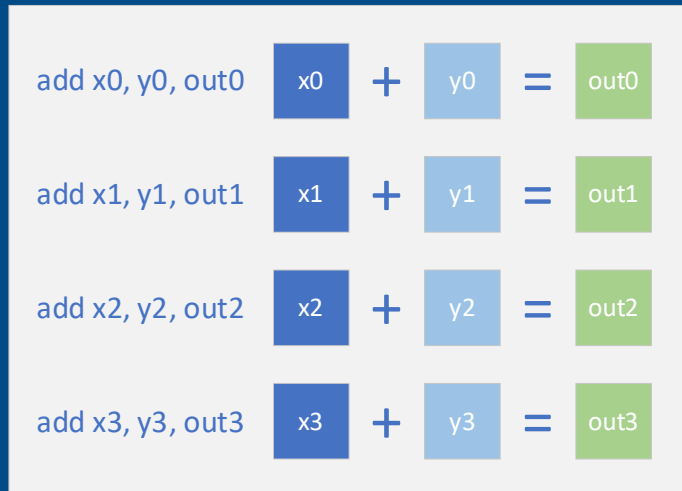
`std::simd` in the
real world

What is SIMD?

What can the hardware do, and how do we access it?

Scalar versus parallel – single-instruction, multiple-data

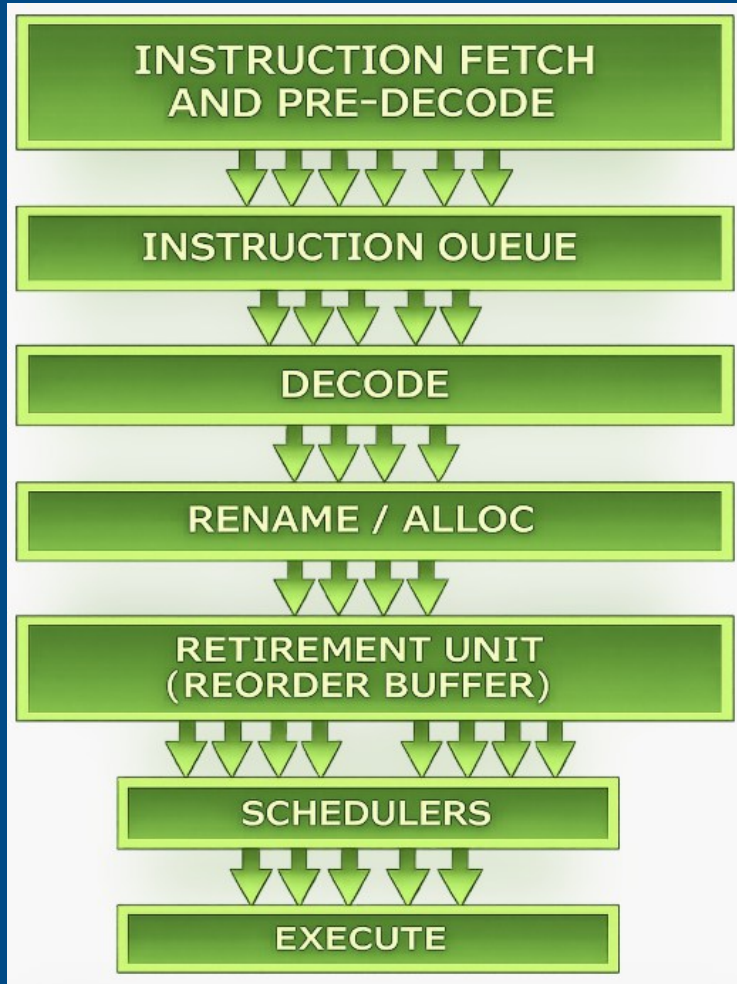
```
auto sum_arrays(const float* x, const float* y, float* out) {  
    for (int i = 0; i < 2048; ++i)  
        out[i] = x[i] + y[i];  
}
```



Left: 4 instructions, 4 adds

Right: 4 instructions, 16 adds! 4x faster.

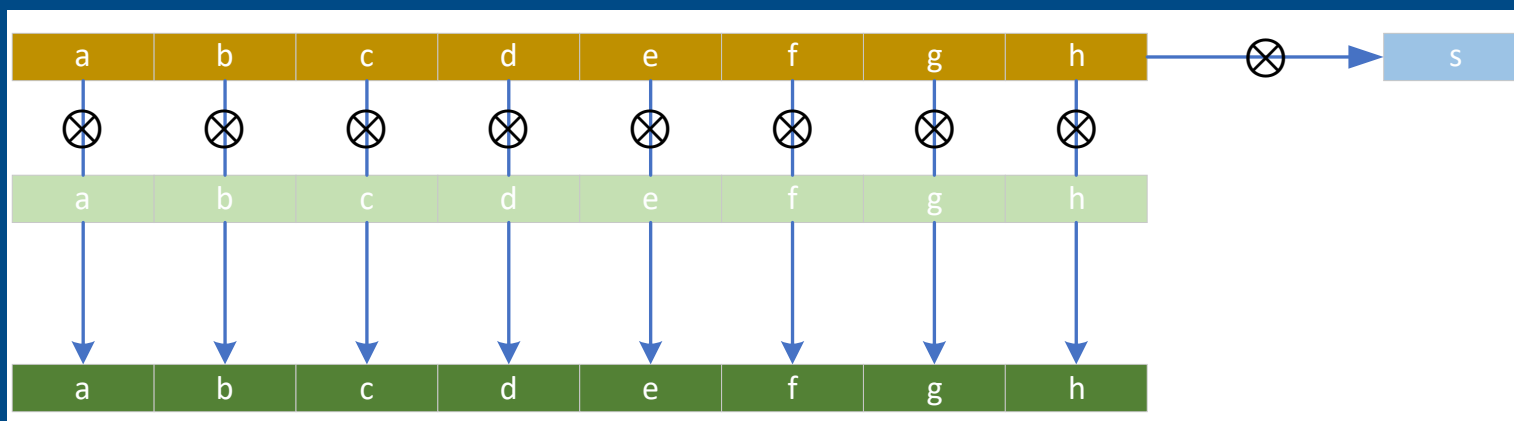
Why SIMD exists — amortizing the pipeline tax



- Every instruction pays a cost:
 - fetch, decode, schedule, speculation, retire
- SIMD amortizes that cost by doing N operations for every instruction, drastically improving performance.
- SIMD works best with packed, contiguous data, naturally encouraging efficient use of cache and memory bandwidth
- The hardware already provides the SIMD capability. The challenge has always been making it accessible to programmers.

Modes of operations

- There are three major dimensions in which vector instructions can execute:



- Vertical/map:** Each element generates a new element
- Horizontal/reduce:** Each element generates a new element
- Shuffle/permute:** reorder the elements

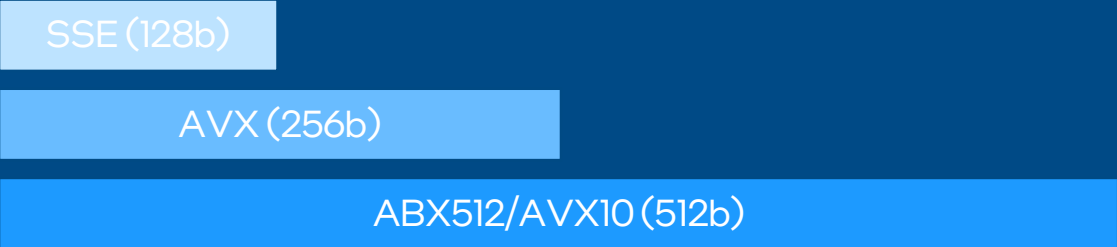
What SIMD can do — operations, widths, masking

Float	Float	Float	Float	4x 32-bit float
Double		Double		2x 64-bit double
B	B	B	B	16x 8-bit byte
short	short	short	short	8x 16-bit short
int	int	int	int	4x 32bit integer
long long		long long		2x 64bit long

The register can be divided up into different numbers and sizes of elements.

	Add	Sub	Mul	Div	Shift
16 x 8-bit int					
8 x 16-bit int					
4 x 32-bit int					
4 x 32-bit float					
2 x 64-bit float					

Not all data types are equal – some lack support (SSE)



Registers have been getting wider over time, increasing efficiency, but also power and complexity.



Masks can be used to deactivate some lanes.

Auto-vectorising

- Auto-vectorisation is able to detect and recover parallelism from sequential code:

```
auto sum_arrays(const float* x,
               const float* y,
               float* out) {
    for (int i = 0; i < 2048; ++i)
        out[i] = x[i] + y[i];
}
```

```
.L4:
    vmovups ymm1, YMMWORD PTR [rbp+4112+rax]
    vaddps ymm0, ymm1, YMMWORD PTR [rbp+16+rax]
    add rax, 32
    vmovaps YMMWORD PTR [rsp-32], ymm1
    vmovups YMMWORD PTR [rax-32+rdi], ymm0
    cmp rax, 4096
    jne .L4
```

- Auto-vectorisation doesn't always work - easy to miss opportunities
- Forcing the user to write sequential representations of parallel code is nasty
- Marking up with pragmas/attributes/hints is not a good solution
- In 25 years of HPC, I have never relied on auto-vectorisation:
 - guaranteed predictable performance through intrinsics is far more valuable.

Accessing simd hardware – compiler features

Explicit use of intrinsics or builtins allows parallelism to be directly expressed.

```
__m512i doSomething(__m512 value)
{
    constexpr __m512 k = _mm512_set1_ps(-0.0);
    const auto s0 = _mm512_andnot_epi16(k,
        _mm512_castps_si512(value));
    const auto s1 = _mm512_mul_ps(_mm512_set1_ps(2.0), s0);
    const auto t0 = _mm512_add_ps(_mm512_set1_ps(1.0), s1);

    const auto km = _mm512_cmp_mask_ps(t0,
        _mm512_set1_ps(threshold), _MM_CMP_GT);
    const auto t1 = _mm512_blend_ps(km, t0, t1);

    return _mm512_cvtps_epi32(t1, 0);
}
```

Intrinsics – verbose, non-portable, and obfuscated

```
v16i doSomething(v16f value) {
    // abs: clear sign bit
    v16u bits = (v16u)value & 0x7FFFFFFF;
    v16f s0 = (v16f)bits;

    // scale and offset
    v16f t0 = s0 * 2.0f + 1.0f;

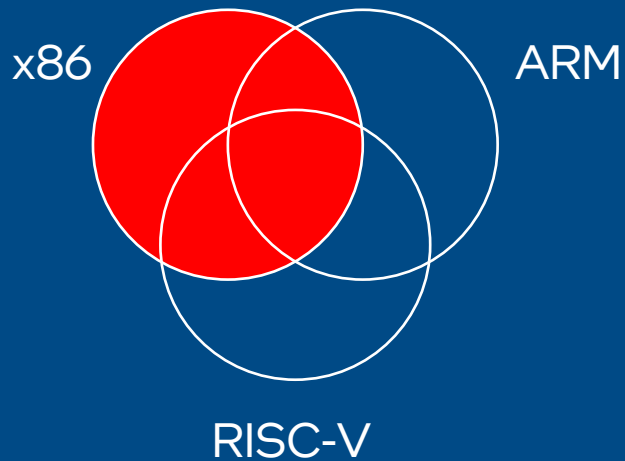
    // blend: keep t0 where > threshold, else zero
    v16f t1 = t0 > value[0] ? t0 : (v16f){};

    // convert to int
    return __builtin_convertvector(t1, v16i);
}
```

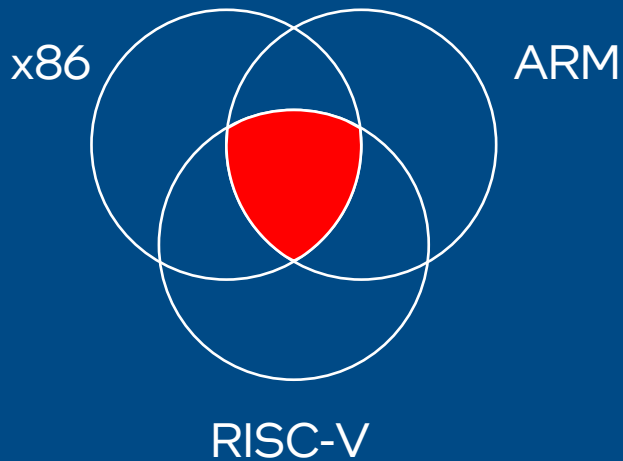
Compiler builtins – clearer and more portable (though not perfect)

Accessing hardware – abstraction libraries

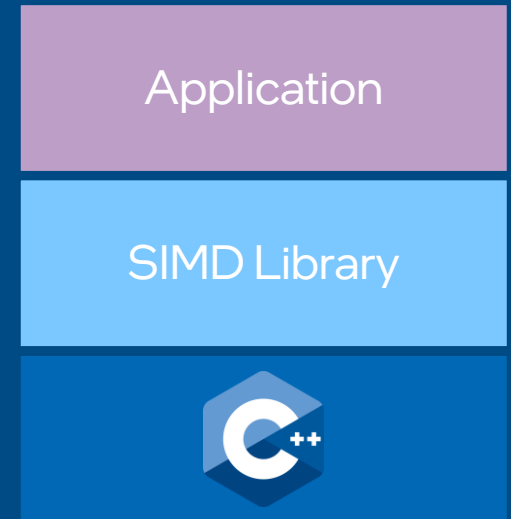
- Many excellent, and widely used libraries available (VCL, Highway, EVE, dozens more)



Exposes target specific features. Not portable.



Portability through `intersection`. Only support features available everywhere.



Libraries are not part of the C++ Standard

Comparison of access techniques

Approach	Strength	Limitation
Auto-vectorization	Ordinary code	Unpredictable vectorisation.
Pragmas and hints	Few code changes	Limited vocabulary
Compiler vector types / builtins	Portable extensions	Compiler-specific, incomplete model
ISA intrinsics	Precise high-performance control	Not portable
SIMD abstraction libraries	Productive APIs with mature optimization	Different APIs, dependencies and behaviours.

Enter: `std::simd`

The library allows portable parallel code to be expressed, which the compiler maps to the target.

Overview of `std::simd`

What C++26 and beyond brings...

Feel the `std::simd` difference

Before

```
__m512i doSomething(__m512 value)
{
    constexpr __m512 k = _mm512_set1_ps(-0.0);
    const auto s0 = _mm512_andnot_epi32(k,
        _mm512_castps_si512(value));
    const auto s1 = _mm512_mul_ps(_mm512_set1_ps(2.0), s0);
    const auto t0 = _mm512_add_ps(_mm512_set1_ps(1.0), s1);

    const auto km = _mm512_cmp_mask_ps(t0,
        _mm512_set1_ps(threshold), _MM_CMP_GT);
    const auto t1 = _mm512_blend_ps(km, t0, t1);

    return _mm512_cvtps_epi32(t1, 0);
}
```

After

```
auto doSomething(simd::vec<float> value)
{
    auto t0 = abs(value) * 2 + 1;

    auto t1 = select(t0 >= value[0], t0);

    return simd::vec<int32_t>(t1);
}
```

Democratises: **Any C++ programmer can express data-parallelism**, not just SIMD experts

Simplicity and readability – `std::simd` style

Native vec of floats,
chosen at compile time

Operator
overloading and
scalar promotion

Overloads for standard
libraries (in std)

New simd-specific
functions to express
common patterns

Conversion to a different
element type

Element indexing

```
auto doSomething(simd::vec<float> value)
{
    auto t0 = abs(value) * 2 + 1;
    auto t1 = select(t0 >= value[0], t0);
    return simd::vec<int32_t>(t1);
}
```

```
_m512i doSomething(_m512 value)
{
    constexpr _m512 k = _mm512_set1_ps(-0.0);
    const auto s0 = _mm512_andnot_epi16(k,
        _mm512_castps_si512(value));
    const auto s1 = _mm512_mul_ps(_mm512_set1_ps(2.0), s0);
    const auto t0 = _mm512_add_ps(_mm512_set1_ps(1.0), s1);

    const auto km = _mm512_cmp_mask_ps(t0,
        _mm512_set1_ps(threshold), _MM_CMP_GT);
    const auto t1 = _mm512_blend_ps(km, t0, t1);

    return _mm512_cvtps_epi32(t1, 0);
}
```

Hardware portability

```
auto doSomething(vec<float> value)
{
    auto t0 = abs(value) * 2 + 1;

    auto t1 = select(t0 >= threshold, t0);

    return simd::vec<int32_t>(t1);
}
```

```
doSomething(simd<float, simd_abi::fixed_size<4>> value)
{
    xorps    xmm2, xmm2
    movaps   xmm3, xmmword ptr [rip + 0]
    movaps   xmm1, xmm0
    cmpltps  xmm0, xmm2
    xorps    xmm3, xmm1
    blendvps xmm1, xmm3, xmm0
    movaps   xmm0, xmmword ptr [rip + 0]
    addps    xmm1, xmm1
    addps    xmm1, xmmword ptr [rip + 0]
    cmpleps  xmm0, xmm1
    andps    xmm0, xmm1
    cvttps2dq xmm0, xmm0
    ret
}
```

Goldmont Atom

```
doSomething(simd<float, simd_abi::fixed_size<8>> value)
{
    vxorps   xmm1, xmm1, xmm1
    vcmltps   ymm1, ymm0, ymm1
    vbroadcastss ymm2, dword ptr [rip + 0]
    vxorps   ymm2, ymm0, ymm2
    vblendvps ymm0, ymm0, ymm2, ymm1
    vbroadcastss ymm1, dword ptr [rip + 0]
    vbroadcastss ymm2, dword ptr [rip + 0]
    vfmadd231ps ymm2, ymm0, ymm1
    vbroadcastss ymm0, dword ptr [rip + 0]
    vcmpleps  ymm0, ymm0, ymm2
    vandps    ymm0, ymm0, ymm2
    vcvttps2dq ymm0, ymm0
    ret
}
```

Broadwell

```
doSomething(simd<float, simd_abi::fixed_size<16>> value)
{
    vxorps   xmm1, xmm1, xmm1
    vcmltps   k1, zmm0, zmm1
    vxorps   zmm0 {k1}, zmm0, dword ptr [rip + 0]
    vbroadcastss zmm1, dword ptr [rip + 0]
    vfmadd213ps zmm1, zmm0, dword ptr [rip + 0]
    vcmpgeps  k1, zmm1, dword ptr [rip + 0]
    vmovaps   zmm0 {k1} {z}, zmm1
    vcvttps2dq zmm0, zmm0
    ret
}
```

Skylake

Recompiling for different native register sizes. The width does not change at run-time.

Source portability (generics)

```
template<class T, int N>
auto doSomething(simd::vec<T, N> value)
{
    auto t0 = abs(value) * 2 + 1;

    auto t1 = select(t0 >= value[0], t0);

    return simd::vec<int32_t, N>(t1);
}
```

<int, 4>

```
fn(xvec::simd::basic_vec<int, xvec::simd::simd_fixed_size_abi<4>
    vpabsd xmm1, xmm0
    vpaddb xmm1, xmm1, xmm1
    vpord xmm1, xmm1, dword ptr [rip + .LCPI0_0]{1to4}
    vpbroadcastd xmm2, xmm0
    vpcmpgtd k1, xmm2, xmm1
    vmovdqa32 xmm1 {k1}, xmm0
    vmovdqa xmm0, xmm1
    ret
```

<float, 16>

```
fn(xvec::simd::basic_vec<float, xvec::simd::simd_fixed_size_abi<16>
    vandps zmm1, zmm0, dword ptr [rip + .LCPI1_0]{1to16}
    vbroadcastss zmm2, dword ptr [rip + .LCPI1_1] # zmm2 = [2
    vfmadd213ps zmm2, zmm1, dword ptr [rip + .LCPI1_2]{1to16}
    vbroadcastss zmm1, xmm0
    vcmpleps k1, zmm1, zmm2
    vmovaps zmm0 {k1}, zmm2
    vcvttss2dq zmm0, zmm0
    ret
```

<short, 12>

```
fn(xvec::simd::basic_vec<short, xvec::simd::simd_fixed_size_abi<12>
    vpabsb ymm1, ymm0
    vpaddw ymm1, ymm1, ymm1
    vpord ymm1, ymm1, dword ptr [rip + .LCPI2_0]{1to12}
    vpbroadcastw ymm2, xmm0
    vpcmpnlw k1, ymm1, ymm2
    vmovdqu16 ymm0 {k1}, ymm1
    vpmovsxbd zmm0, ymm0
    ret
```

Use one algorithm for any suitable data type or number of elements.

Expressing intent over implementation

- Imperative programming, describe **how to achieve** something

```
const auto as64 = _mm512_castps_epi64(values); // Treat as 64-bit units
const auto t = _mm512_cvtepi64_epi32(as64);    // Pull out the lower 32-bits
const auto evens = _mm256_castepi32_ps(t);     // Treat as float again.
```

- `std::simd` describes the intent of **what to achieve**

```
auto evens = permute(values, [](auto idx) { return idx * 2; });
```

- `std::simd` is a clean interface for accurately describing what to achieve, which allows the compiler/library to implement that intent on a range of targets.

Construction and conversion of simd vec values

- Generator-based construction:

```
// LUTs can be defined using lambda generators
constexpr simd::vec<int, 16> lut0([](auto i) { return i * 7 % 3; });
```

- iota-based (C++29)

```
// ...or direct operations on iota (0, 1, 2, ...)
constexpr auto lut1 = simd::iota<simd::vec<int>> * 7 % 3;
```

- From spans:

```
std::span<float, 8> sf = ...;
simd::basic_vec v = sf;
```

- Modern C++ brings new conversion safety guarantees:

```
simd::vec<short> x = lut0; // Error: Not value-preserving
simd::vec<short, 16> x = simd::vec<short, 16>(lut0); // use explicit to force
```

Masks

```
simd::vec<float> x = ...;

// Create a mask from relational operator
// classification, generator, etc.
auto m0 = x > 34.f;
auto m1 = isinf(x);
auto m2 = simd::mask<float>([](auto idx) { return idx % 2 == 0; });
simd::mask<float> m3 = 0b10100101;

// Combine masks using operators
auto m4 = m0 & m1 | m2;

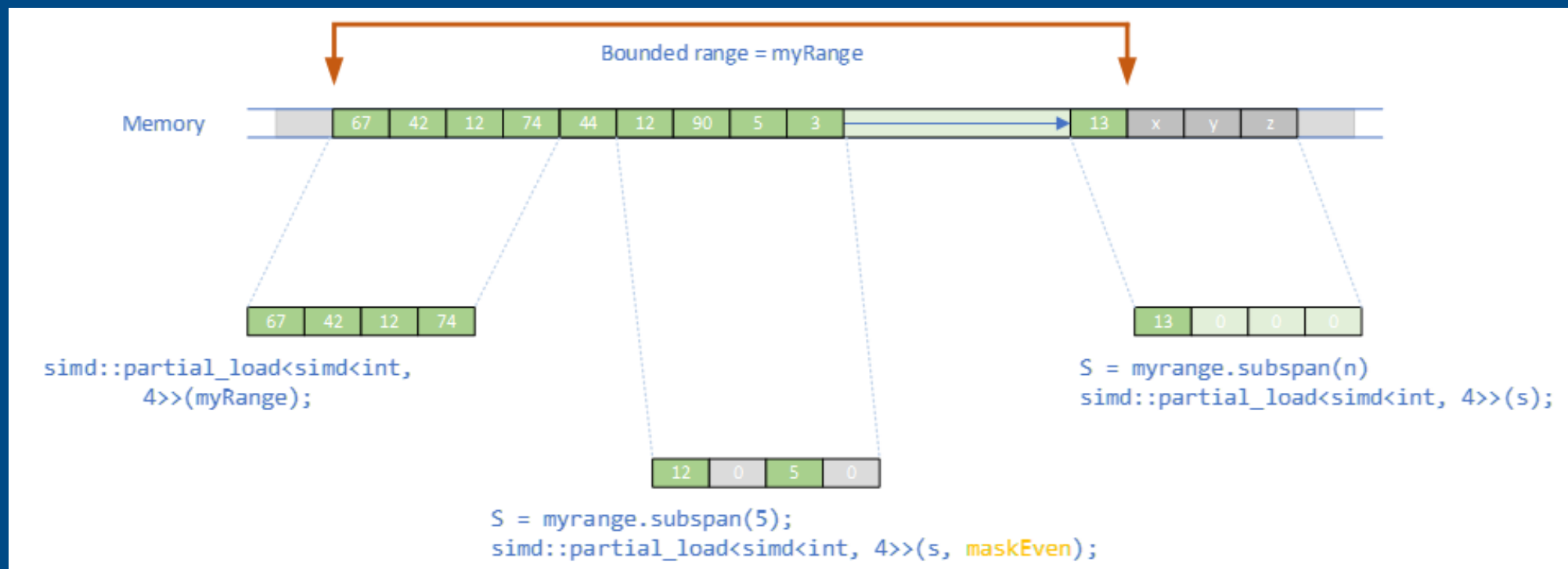
// Use masks to drive active elements in operations
auto conditionalAdd = select(isinf(x), x, x + 1.0f);
auto squashed = compress(x, x > 0);

// Extract properties of masks
auto numPositiveValues = reduce_count(x > 0);
```

Portable across wide or dense masks

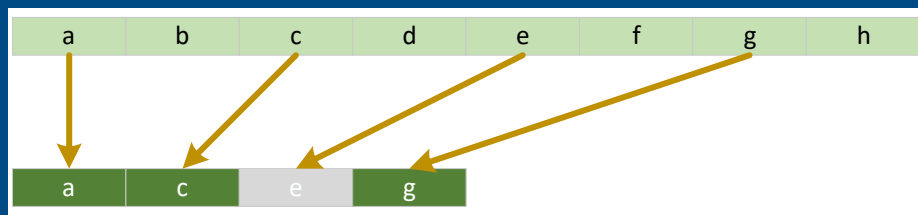
Modern C++ brings new safety guarantees

- Modern C++ provides bounds-aware interfaces that make the size requirement explicit.
 - raw pointers/iterators discouraged



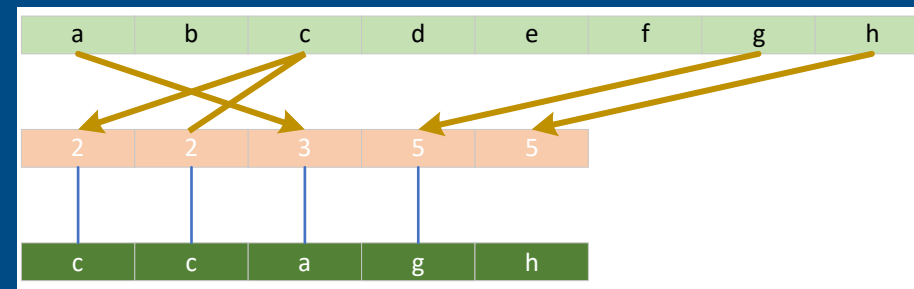
- (for performance, there is an unchecked version too)

Permutations

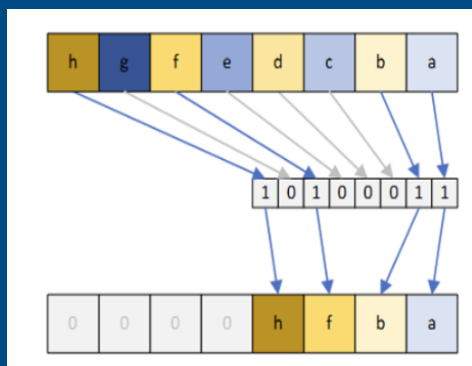


```
permute<4>([](auto i) { return i * 2; })
```

Static permute



Dynamic permute (one vec/mask by another vec)

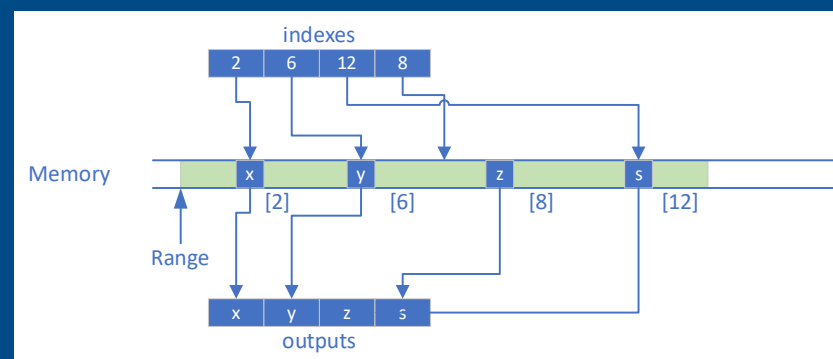


```
simd::vec<float> x;
```

```
simd::mask<float> m;
```

```
auto out = compress(x, m);
```

Compress/expand using masks



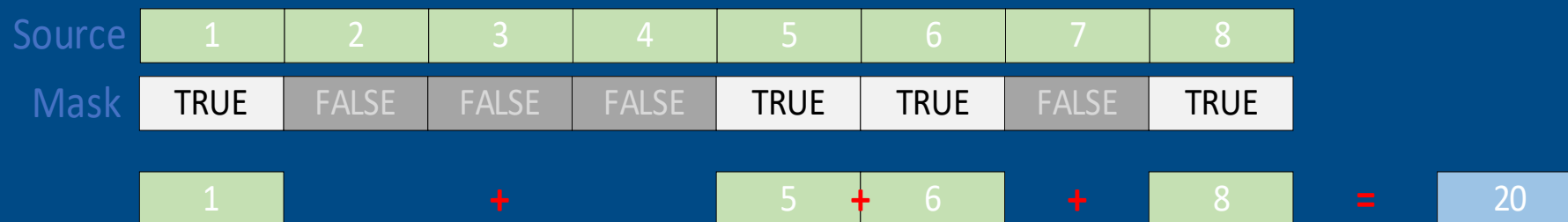
Memory gather/scatter

Reductions – horizontal operations

- Apply an operation between elements of one vector, to generate a single result (e.g., addition):



- Use masks to apply to specific elements:



- Example:

```
float dotprod = reduce(v * v, std::plus<>{});  
int  smallest = reduce_min(v);
```

xvec: Intel's `std::simd` implementation

You've seen what the library does. Here's how it's built.

From proposal to implementation

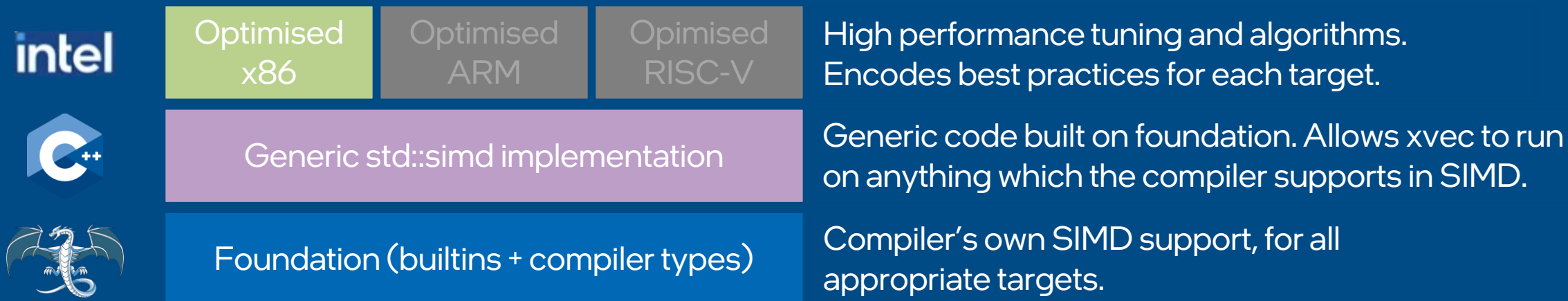
- Intel was working on its own extensible vectorization library, but in 2022 we started collaborating with Matthias Kretz on `std::simd`
- This collaboration resulted in multiple accepted proposals to C++26, including permutations, SIMD and complex numbers, SIMD `<bit>` header extensions
- `xvec` is a complete implementation of the C++26 `std::simd` model

xvec

- Built on compiler SIMD support, with optional target-specific optimization
- Designed to be:
 - portable across compiler-supported targets;
 - performant without requiring every operation to be hand-written;
 - extensible by ISA experts;
- Used as implementation experience for standardization

xvec – Intel's `std::simd` implementation

xvec is built from 3 layers:



- By building on generic compiler SIMD support, xvec works on any target, with optimisation and tuning being an optional add-on.
- Full details at <https://www.youtube.com/watch?v=viiKWsooITQ>

Foundation and generic implementation layer

```
typedef float v8f __attribute__((vector_size(32)));
```

```
v8f process(v8f a, v8f b) {  
    // operator overloading  
    v8f sum = a + b;  
  
    // Builtin simd functions  
    v8f clamped = __builtin_elementwise_min(sum, (v8f){1,1,1,1,1,1,1,1});  
  
    auto smallest = __builtin_reduce_min(clamped);  
  
    // builtin conversion - float to int  
    auto converted =  
        __builtin_convertvector(clamped,  
                                int __attribute__((vector_size(32))));  
  
    // Builtin permutation  
    return __builtin_shufflevector(converted, converted,  
                                   1,0, 3,2, 5,4, 7,6) + smallest; // swap pairs  
}
```

vaddps ymm0, ymm0, ymm1

vminps ymm0, ymm0, ymm1

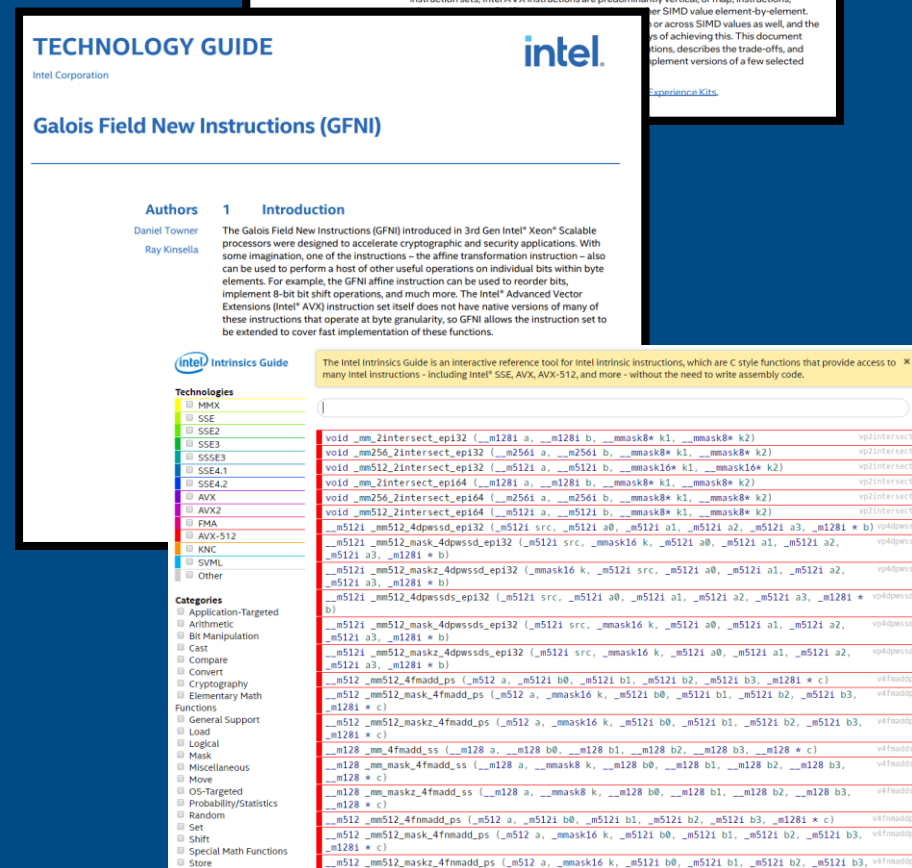
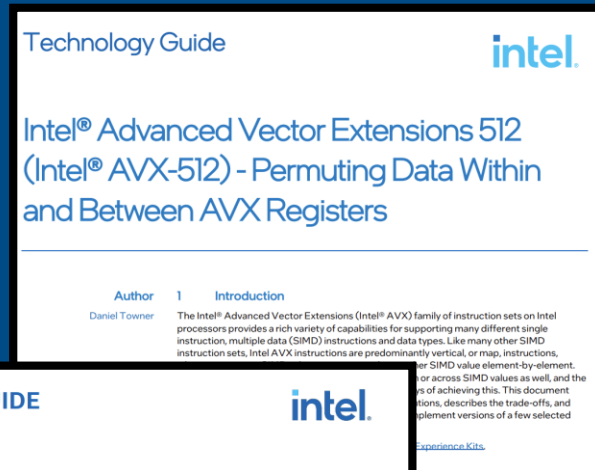
```
vextractf128    xmm1, ymm0, 1  
vshufps xmm2, xmm1, xmm1, 255  
vshufpd xmm3, xmm1, xmm1, 1  
vmovshdup      xmm4, xmm1  
vshufps xmm5, xmm0, xmm0, 255  
vshufpd xmm6, xmm0, xmm0, 1  
vmovshdup      xmm7, xmm0  
vminss xmm8, xmm7, xmm0  
vcmpunordss    xmm0, xmm0, xmm0  
vblendvps      xmm0, xmm8, xmm7, xmm0  
vcmpunordss    xmm7, xmm0, xmm0
```

vshufps ymm0, ymm0, ymm0, 177

- The generic implementation of `std::simd` runs entirely on top of the foundation, which gives a high-degree of compiler + target independence.
- As the compiler supports new targets or features, the generic implementation automatically extends.

Optional target-specific layer

- Overrides generic functions and features with target-specific optimisations
- Knows about target-specific timing, performance, instructions/intrinsics and features.
- Encodes best practice from target experts.
- Can be used to optimise xvec for a new target



User defined types

- Useful for other domains
- Integrates with all features currently permitted.
- Opens up SIMD for use in unusual places (e.g., packet processing, where compress/expand/gather/scatter and others are genuinely useful)
- Ideally, should just work if a type fulfil the defined set of requirements (proposed by P2964R5)

User defined types example

```
struct point2d
{
    float x;
    float y;

    // operations
};

// The code should just work based on current state of P2964R5
simd::vec<point2d> obj{ /* ... */ };

auto distance(vec<point2d, 8> x, vec<point2d, 8> y) {
    return x + y;
}
```

Parallel algorithms and `std::simd`

- Parallel algorithms API today

```
// x is int
for_each(execution::seq, data.begin(), data.end(), [](auto& x) { x *= x; });
```

- Parallel algorithms idea with `std::simd` (P0350R4)

```
// x is std::simd::vec<int>
for_each(execution::simd, data.begin(), data.end(), [](auto& x) { x *= x; });
```

- Neither of execution policies changes algorithm lambdas parameter type today
- Execution seems to be orthogonal to data packing to `std::simd` (e.g., `execution::par` and `execution::simd` is a valid combination)
- Unclear interaction with `execution::[par_]unseq`

Moving development to open source

- Build an ecosystem around `xvec` and `std::simd`
- Provide an implementation and adoption experience for `std::simd` proposals
- Provide non-standard extensions to `std::simd`

Current state

- `std::simd` is in the working draft (partially implemented in GCC)

Plan to open source `xvec` here: <https://github.com/intel/xvec>

`std::simd` in the real world

A pragmatic look at real-world scenarios

Democratizing SIMD: Empowering the Domain Expert

The SIMD bottleneck

- Code requires:
 - Domain expertise
 - SIMD expertise
- Rare to have both
- Non-portable, fragile code

The shift to `std::simd`

- Focus on intent
- Rapid iteration and clean programming model
- Let the domain expert describe the problem, and the compiler/library generate the code

The outcome

- Anyone can write code which achieves reasonably speed-up
- The code might well be fast enough for the application.
- If the code isn't fast enough, it is a clean specification of the problem and the available parallelism which a SIMD expert can analyse and tune.

- All in the context of idiomatic standard C++.

4 sources of performance (and losses)

`a * b`

```
vmulps zmm0, zmm1, zmm0
```

1:1 – No room for perf loss

`__builtin_reduce(v)`

1:N – Limited perf loss,
fixable in new compiler
version.

```
vextracti64x4 ymm1, zmm0, 1
vpaddq zmm0, zmm0, zmm1
vextracti128 xmm1, ymm0, 1
vpaddq xmm0, xmm0, xmm1
vpshufd xmm1, xmm0, 238
vpaddq xmm0, xmm0, xmm1
vpshufd xmm1, xmm0, 85
vpaddq xmm0, xmm0, xmm1
vmovd eax, xmm0
```

`bit_reverse(a)`

Function: Relies on
implementation
QOI.

```
vpsrlw ymm1, ymm0, 4
vpbroadcastb ymm2, byte ptr [ri
vpand ymm1, ymm1, ymm2
vbroadcasti128 ymm3, xmmword ptr
vpshufb ymm1, ymm3, ymm1
vpand ymm0, ymm0, ymm2
vbroadcasti128 ymm2, xmmword ptr
vpshufb ymm0, ymm2, ymm0
vbroadcasti128 ymm2, xmmword ptr
vpshufb ymm0, ymm0, ymm2
```

```
stride<3>(rebind_cast<uint8_t>(a))
// rebind_cast<uint8_t>(stride<3>(a))
```

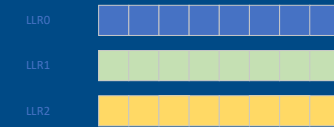
Composition: How does
the programmer think?
Solved by analysis and
tuning.

```
vpmovdb xmm0, ymm0
vpshufb xmm0, xmm0, XMMWORD
```

```
vmovdqa ymm1, YMMWORD PTR
vpermd ymm0, ymm1, ymm0
vpmovdb xmm0, xmm0
```

Our target was to get within 10% of the performance of a good intrinsic baseline

Performance example - interleave



```
inline
void InterleaveComplexLLRs(F32vec8 srcA, F32vec8 srcB, F32vec8 *result)
{
    //_mm256_castps_pd
    const auto up_lo = _mm256_unpacklo_pd(_mm256_castps_pd(srcA), _mm256_castps_pd(srcB));
    const auto up_hi = _mm256_unpackhi_pd(_mm256_castps_pd(srcA), _mm256_castps_pd(srcB));
}
```

```
constexpr int imm8Lo = 0b00110000;
constexpr int imm8Hi = 0b00110001;
```

```
result[0] = _mm256_castpd_ps(_mm256_
result[1] = _mm256_castpd_ps(_mm256_
}
```

//Interleave for 3 complex LLRs

```
inline
void InterleaveComplexLLRs(F32vec8 srcA,
```

```
{
    const auto pA = _mm256_permute4x64_pd(
    const auto pC = _mm256_permute4x64_pd(
```

```
const auto upPerm = _mm256_unpacklo_pd(
```

```
const auto upAB = _mm256_unpacklo_pd(
const auto upBC = _mm256_unpackhi_pd(
```

```
constexpr char blendMask = 0b00001100,
res
```

inline

```
void InterleaveComplexLLRs(F32vec16 srcA, F32vec16 srcB, F32vec16 *result)
```

```
{
```

```
const auto k_lower = _mm512_setr_epi64(0, 8, 1, 9, 2, 10, 3, 11);
```

```
const auto k_upper = _mm512_setr_epi64(4, 12, 5, 13, 6, 14, 7, 15);
```

```
result[0] = _mm512_castpd_ps(_mm512_
k_lower, _mm512_castps_pd(srcB) );
result[1] = _mm512_castpd_ps(_mm512_
k_upper, _mm512_castps_pd(srcB) );
}
```

//Interleave for 3 complex LLRs

inline

```
void InterleaveComplexLLRs(F32vec16
*result)
```

```
{
```

```
const auto srcA_d = _mm512_castpd(
```

```
const auto srcB_d = _mm512_castpd(
```

```
const auto srcC_d = _mm512_castpd(
```

```
const auto k_idx0 = _mm512_setr_
```

```
const auto k_idx1 = _mm512_setr_
```

```
const auto k_idx2 = _mm512_setr_
```

```
const auto permA = _mm512_permut
```

```
const auto permB = _mm512_permut
```

```
const auto permC = _mm512_permut
```

```
const __mmask8 k_blendMask = 0b0
```

```
inline void InterleaveComplexLLRs(F16vec32 srcA, F16vec32 srcB, F16vec32 *result)
```

```
{
```

```
const auto k_permLo = _mm512_setr_epi32(0,16,1,17,2,18,3,19,
4,20,5,21,6,22,7,23);
```

```
const auto k_permHi = _mm512_setr_epi32(8,24,9,25,10,26,11,27,
12,28,13,29,14,30,15,31);
```

```
const auto result_0 = _mm512_permutex2var_epi32(_mm512_castph_si512(srcA),
k_permLo, _mm512_castph_si512(srcB));
```

```
const auto result_1 = _mm512_permutex2var_epi32(_mm512_castph_si512(srcA),
k_permHi, _mm512_castph_si512(srcB));
```

```
result[0] = _mm512_castsi512_ph(result_0);
```

```
result[1] = _mm512_castsi512_ph(result_1);
```

```
}
```

```
inline void InterleaveComplexLLRs(F16vec32 srcA, F16vec32 srcB,
F16vec32 srcC, F16vec32 *result)
```

```
{
```

```
const auto k_idxAB = _mm512_setr_epi32(0, 16, 27, 1, 17, 28, 2, 18,
29, 3, 19, 30, 4,20, 31, 5 );
```

```
const auto k_idxAC = _mm512_setr_epi32(10, 27, 22, 11, 28, 23, 12,
29, 24, 13, 30, 25, 14, 31, 26, 15 );
```

```
const auto k_idxBC = _mm512_setr_epi32(5, 21, 16, 6, 22, 17, 7, 23,
18, 8, 24, 19, 9, 25, 20, 10 );
```

300+ LOC

Interleave cont...

```
template<typename T, typename... SIMD>
inline
void transpose(std::span<std::complex<T>>* result,
              SIMD... l1r_groups)
{
    const auto t = cat(l1r_groups...);

    constexpr auto numGroups = sizeof...(l1r_groups);
    constexpr auto numPerGroup = decltype(t)::size() / numGroups;

    const auto rt = permute(t, [](auto idx) {
        return (idx / numGroups) + (idx % numGroups) * numPerGroup;
    });

    partial_store(rt, result);
}
```

- Typically as fast as the hand-written code.
- Sometimes it is even faster because the compiler knows more tricks than me!
- And it runs on any target!

Performance example - hand-crafted functions

- Pop count can be implemented in one of several ways:

```
basic_vec::popcount(auto v) {  
    return details::popcount(vendor, v);  
}
```

```
template<vec_integral V>  
auto popcount(generic_tag, V v) {  
    // Use a generic algorithm for integral types  
    // e.g., Wilkes-Wheeler-Gill algorithm  
}
```

```
template<vec_integral V>  
requires (sizeof(typename V::value_type) == 1)  
auto popcount(x86_avx_tag, V v) {  
    // Use custom version for bytes on AVX  
}
```

```
template<vec_integral V>  
auto popcount(x86_avxsnc_tag, V v) {  
    return __mm512_popcount_epiXX(v);  
}
```

```
vpbroadcastb    ymm1, byte ptr  
vpand          ymm2, ymm0, ymm1  
vbroadcasti128  ymm3, xmmword  
vpshufb        ymm2, ymm3, ymm2  
vpsrlw         ymm0, ymm0, 4  
vpand          ymm0, ymm0, ymm1  
vpshufb        ymm0, ymm3, ymm0  
vpaddb         ymm0, ymm0, ymm2  
vpsrld         ymm1, ymm0, 16
```

One example – Treat each nibble as an index into a LUT of popcounts, and sum nibbles.

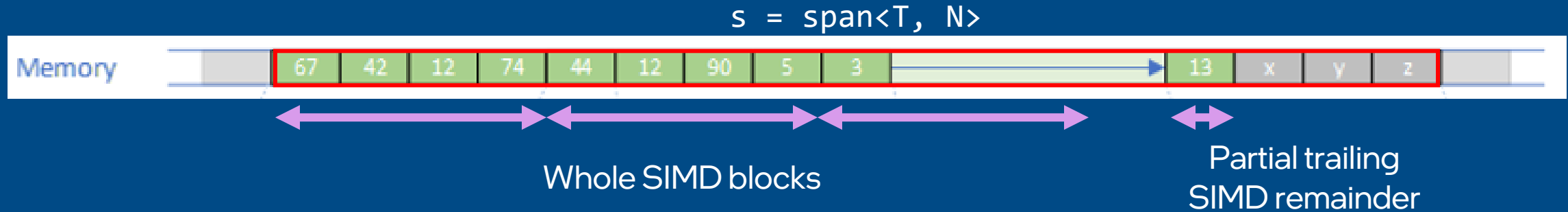
Key takeaway – write code using `std::simd` functions and get the best possible library implementation!

SIMD generic programming: width-agnostic code

- Consider:
 - The algorithm doesn't care how many values are processed: one, a few, many.
 - Logical limit of "any-width" is a scalar.
 - Write and test code for scalar (quick), extend to SIMD for performance
- We want to write width-agnostic code.
- `std::simd` allows this by providing scalar versions of its functions:
 - `select` (ternary operator?), `any_of`, `reduce`, `math`, `bit`, and more...
- Not perfect though – more needed in C++29:
 - `std::numeric_limits<V>`
 - Load/store/gather/scatter/etc.

```
template<class V> V relu(V x) {  
    return max(x, V(0));  
}  
  
float      y = relu(x);    // one value  
vec<float> ys = relu(xs);  // every lane
```

Processing spans



```
auto process = [...](...){...};

for (int i = 0; i < totalBlocks; ++i)
{
    auto ss = s.subspan(i * vec<T>::size);

    auto e = simd::partial_load<vec<T>>(ss);

    process(e);
}
```

```
for (int i = 0; i < numWholeBlocks; ++i)
{
    auto ss = s.subspan(i * vec<T>::size);

    auto e = simd::unchecked_load<vec<T>>(ss);

    process(e);
}

if (remainder)
{
    auto e = simd::partial_load<vec<T>>(s.last(remainder));
    auto m = simd::mask_from_count<vec<T>>(remainder);

    process(e, m);
}
```

Intrinsic integration

- Despite our desire for portability, sometimes you just need intrinsics (e.g., special target-specific features, like VNNI, GFNI, and ACE).

```
vec<float> vf = ...;

// std::simd to builtin
__m256 t0 = __m256(vf);

// Call intrinsic
auto t1 = _mm_dpps(t0);

// Back to simd
auto r =
    std::vec<float>(t1);
```

C++26

- Direct ctor/convertor for interaction with builtins with native sizes.

```
auto dpps = [](auto vs) {
    // Do intrinsic.
}

...

vec<float, 19> v = ...;

auto t =
    simd::chunked_invoke(dpps, v);
// r0 = dpps(extract<0, 8>(v))
// r1 = dpps(extract<8, 16>(v));
// r2 = dpps(extract<16, 19>(v));
// t = cat(r0, r1, r2);
```

C++29

- Handle arbitrary widths

```
namespace xvec::x86 {
    auto gf2p8affine(vec_of<uint8_t> b,
                    uint64_t m);

    // vnni
    // ace
    // fma variants
    // etc.
}
```

xvec

- Provide wrappers for common intrinsics.
- (work in progress)

Rebinding – influence beyond `std::simd`

- Element-wise conversion is a common operation:

```
vec<float> vf = ...;  
vec<short> vi0 = vec<short>(vf);
```

- C++26 has a new vocabulary for this: *rebinding*

```
auto vi1 = rebind_t<short, vec<float>>(vf);  
// similar to vec<short, vec<float>::size()>
```

- C++29 proposes `rebind_cast`:

```
auto vi2 = rebind_cast<short>(vf);
```

- `rebind_cast` is useful beyond `simd`:

```
std::array<float, 23> af = ...;  
auto sd = rebind_cast<double>(af);  
// std::array<double, 23>
```

```
std::complex<float> cf = ...;  
auto cd = rebind_cast<double>(cf);  
// std::complex<double>
```

Closing

Wrap-up

- SIMD hardware gives us declarative data-parallelism without sacrificing portability
- It puts high-performance data-parallelism into the hands everyone.
- We've open-sourced xvec: <https://www.github.com/intel/xvec>
- Ruslan and Daniel are here all week – come and talk to us!

Questions